

AI Interaction Log

Author: Bryce **Date:** 2026-09-25 **AI Tool:** Claude (Cowork)

Task / Prompt

Review PR #3 (“Database schema, migrations, seed data, and ERD”, branch `database` → `main`) and check that the code aligns with issue #1 (“[Task] Design and implement database schema”).

Context / Findings

- PR #3 declares Closes #1. Head `database`, base `main`, open, +665/-0 across 8 files: `backend/schema.sql`, `migrate.js`, `seed.js`, `package.json`, `package-lock.json`, `README.md`, `.gitignore`, and `docs/ERD.md`.
- Stack chosen: SQLite via `better-sqlite3`, passwords hashed with `bcryptjs`; resumes stored on disk with path + metadata in the `resumes` table (matches issue’s open decisions).
- Sprint 1 tables (`users`, `profiles`, `resumes`) match issue #1’s column spec exactly. Later-sprint entities (`jobs`, `applications`, `saved_jobs`, `notifications`) are laid out in `docs/ERD.md` as the issue asked (“design now, implement later”).
- Constraints present: PK autoincrement, email UNIQUE COLLATE NOCASE, role CHECK enum, NOT NULLs, FKs with ON DELETE CASCADE, `profiles.user_id` UNIQUE (1:1), timestamp defaults. Indexes: explicit `idx_resumes_user_id`; email and `profiles.user_id` are covered by their UNIQUE constraints (auto-indexed).
- Verification: exported the `database` branch to a scratch dir, ran `npm install + npm run db:reset` from a clean checkout — migrate and seed both ran without errors. Ran a constraint test script: unique-email (case-insensitive), role CHECK, FK enforcement, one-profile-per-user, and ON DELETE CASCADE all behaved correctly. Re-running the seed is idempotent (stays at 2 users via INSERT OR IGNORE).

AI Output / Actions

- Confirmed PR #3 aligns with issue #1: all Sprint 1 scope, schema-design-now items, DBMS decision + documentation, ERD in `/docs`, migration + seed scripts, constraints, indexes, seed data, and setup docs are satisfied.
- Minor notes raised (non-blocking): `updated_at` columns are set on insert but have no UPDATE trigger, so they won’t refresh unless the app sets them; schema uses `CREATE TABLE IF NOT EXISTS` (single-file schema, re-created via `--reset`) rather than incremental versioned migrations; DBMS decision is documented in `backend/README.md` rather than the root README; the last DoD item (auth/profile/resume endpoints read/write against the schema) can’t be confirmed until the API PR lands.

Decision

Bryce: The review lined up with what I expected from issue #1, so I passed the findings along and we moved ahead with the PR. The minor notes (`updated_at` not refreshing, single-file schema instead of versioned migrations, DBMS decision documented in the backend README) were left as non-blocking — worth mentioning to the team but not enough to hold the merge.

Reflection

Bryce: This was my first real code review on someone else's backend work, and running the branch from a clean checkout to actually confirm the migrations and constraints behaved gave me a lot more confidence than just reading the diff. I picked up on what to look for in a schema — constraints, indexes, cascade behaviour — that I wouldn't have thought to check before.